Lab 1: Getting your hands dirty with basic robotics – DEIS DT8007

Supervisor: Martin Cooney (~ 2 hours)

Today we will do something hands-on with robotics!

Goal: by completing this lab, the goal is that you will, in designing embedded and intelligent systems:

- Have an idea of how to do basic robotics tasks, like connecting to a microcomputer, processing video from a webcamera, and sending messages between different machines (which could be both robots and computers).

All labs in this course relate mainly to the second intended learning outcome in the course syllabus, called "Skills and ability: program an embedded computer implementing intelligent functions for sensing (such as situation and position estimation, comprising calibration, filtering, and fusing signals from different sensors), actuator control, and wireless communication in a distributed system".

Note the labs in this course are separate from the project!--They might sometimes also help you with the project but cover a wider range of different tasks that might not be part of the project.

Approach: to complete the lab, you will require the following:

- Adequate preparation. You should have downloaded this file for the lab from Blackboard and read it briefly before starting, and you should have formed a group. You should have preferably also completed the preparation step of installing Ubuntu and ROS2 on your raspberry pi computer (hereafter "rpi")
- Facilities and Parts. Access to your group's rpi with related paraphernalia. Note, each group should have a rpi 4. (Some groups might have also a rpi 5, depending on circumstances.) You should use your adapter to power the rpi if you can, not a battery, and disconnect it from the arduino/robot. You will also need to work with the rpi somehow (i.e., you will need to have a computer which can ssh into the rpi). There should also be some hdmi cables which can be used to directly hook up your rpis to a monitor. You also will want to be able to access as a group the internet with some computer, a project pc or your own computer, to be able to see tutorials and install software (e.g., by sharing a connection from some other computer, on eduroam or some home network or by making a hotspot with your phone). For networks: We will also have our own router in the A113 project room for this course which will not have internet access based on our IT dept.'s requirement. (SSID: robot4, pwd: weighs67CATS.)

Result:

- A submitted report. After the lab, please submit through Blackboard a short report in pdf format with filename "DEIS_labX_groupY.pdf" including screenshots and any comments. (Replace "X" with the lab number and "Y" with your group number.) Please see our shared deadlines document to see the recommended dates of completion and absolute deadlines for this lab.

You will receive a response by email within 15 working days from the deadline at the latest. Your report will be judged as passing, reasonable but failing, or unsubmitted. Any feedback will typically be short, around a line or two in the case the lab is still failing, that should indicate what remains to be done to attain a passing grade.

Please note:

- If the report is reasonable but failing, you will get a chance to review and resubmit.
- All labs must be submitted to receive a passing grade in the course.
- Keep the lab report reasonably brief. There's no need for lots of words.
- Use of AI tools: Reasonable use of tools in this lab as a helper is okay (like ChatGPT or such to give you advice if you encounter a problem or to check English); just you must declare it.
- Your report should include a table detailing exactly which group member did what. It's perfectly fine to divide up labs so that not all group members work on them, but each group member should at least have seen the submitted report. A table can look like this (with names given and activities explained):

	Member1	Member2	Member3	Member4

Activity 1	50%	50%		
Activity 2		100%		
Activity 3	25%	25%	25%	25%

- **You are expected to do original work. Use of someone else's code/report/words/etc without referencing them is a _serious_ problem which will be reported to the disciplinary board.**

Tasks:

0. Preparation. (1-2 hours? This should be done preferably before coming to the lab.)
Install an operating system and robot software on your SD cards. Follow the instructions below:
https://ubuntu.com/download/raspberry-pi
https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debians.html
For both rpi 4 (and rpi 5 if any group uses one), we use Ubuntu 24.04.3 and ROS 2 Jazzy:
Ubuntu is an awesome/popular/well-supported OS, which is also popular for working with robots; the latest stable build, Ubuntu 24.04 LTS, aka Noble Numbat, was released April 25, 2024, with end of life in April 2036.
ROS is *the* "big" thing roboticists use for controlling robots and sharing code; ROS 2 is the newest kind of ROS, which everything is moving to use. We will use the latest stable version of ROS 2, Jazzy Jalisco, released 23 May 2024, with end of life May 2029, to allow our robots and infrastructure to communicate.
(If you want to and know what you are doing, you can also try to install ROS2 on Raspbian instead of Ubuntu or other, but in general Ubuntu is recommended… Likewise, Ubuntu Desktop and ROS2 desktop versions are recommended, since they are easiest to use when you're starting out and give you the most options, but you could try for example installing a lighter version of Ubuntu like server or core, and a lighter version of ROS2 since we only use it for communication in this course… (Also, you could also try ROS 1 for fun, although it is not needed/used in this course…))
In more detail:
●If using rpi5: ensure the fan is securely attached. The connector should be attached, the paper at the bottom of the thermal pads should have been removed, and the screws should have been pushed through. Have something safe under the rpi 5 to avoid shorts, not your hand or lap or such.
●Copy Ubuntu onto the SD card (RPI Imager or other). When done, slide the SD card into the rpi. Then connect a monitor, keyboard, and mouse.
● Try starting up your rpi. Insert the power cable into the outlet. After a few seconds, you should see text appear. Do the set up. Ensure you choose something you will remember that is secure for your password. Connect to some network with internet, e.g. using an ethernet cable from your home router or a hotspot from your phone or such.
After the first time, you can consider connecting some other way, like with a wired connection to a router with a LAN cable and finding the rpi's IP address by logging into your router, then sshing in (as below) or using vnc.
● When you are connected to an internet connection, ensure everything's nicely updated with "sudo apt update", "sudo apt upgrade".
● Install ROS2 _Jazzy_ (be careful not to install any other version). Copy and paste each line of code one by one carefully. Install also the "optional" devtools, and install the Desktop version (not another version, unless again you know what you are doing).
https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html

1. Try out your raspberry pi. (10 minutes)
This is hopefully the first thing you do on the day of the lab.
●Try connecting to our router/network. (SSID: robot4, pwd: weighs67CATS)
●Try the talker / listener check to make sure ROS 2 is working.
Open two terminals. In one paste in:

```
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_cpp talker
```
In the other paste in:
```
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_py listener
```
Take a screenshot or photo of the output of running the talker and listener and add it to your report.

2. SSH: connecting remotely to a different computer (~20 minutes)
When working with robots, we often need to connect to a remote computer.
SSH is one basic way to do that.
First you might need to install an ssh server.
Connect to a network with internet (like a phone hotspot)
```
sudo apt update && sudo apt upgrade
sudo apt install openssh-server
sudo systemctl enable --now ssh
sudo systemctl status ssh
```
(The last command is not needed/just checks that all is good. You probably won't have to do anything with firewalls, but you can see more info/details below: https://www.how2shout.com/linux/installing-openssh-server-on-ubuntu-24-04-lts-noble-linux/)
●Now that you have your ssh server running on the rpi, let's try to log in with another computer. Ensure your rpi and own computer are connected to the same network. If you are at HH in the E1 project room, you can use our router.
Now get the rpi's ip address. E.g., you can try (1) or (2) below, or both for fun If you want…
Either (1) if the rpi has a monitor attached you can run the command "ip -a" from the command line and look at the IP address for eth0.
Or (2) if you have previously set the rpi to connect automatically to our router, you can login to the router to find/confirm the rpi's IP address. You can just open an internet browser on some computer and then type in 192.168.1.1 or such. The admin id and password are on the bottom of the router. Then click to find the attached devices.
●Use ssh to connect to your raspberry pi: e.g., from ubuntu/linux (assuming you have a client):
```
sudo ssh deis@192.168.1.X
```
(Replace the IP address above with the one you found in the step before.)
On windows you can ssh using Putty (download from http://www.chiark.greenend.org.uk/~sgtatham/putty/download.html and run)
(There are also various ssh apps for smart phones; e.g., years ago I used to use JuiceSSH on android)
Note: you should change your rpi's hostname in /etc/hostname and /etc/hosts to something unique and enjoyable for you, since your rpi will communicate with other rpis and having the same name for all of them could cause confusion.
Take a screenshot or photo of sshing in and add it to your report.

3. Image Processing on Camera Feed (10 minutes)
In this course, your robots are equipped with cameras.
Computer vision with cameras is a fantastically powerful way to allow robots to sense their surroundings.
OpenCV has been a massively popular standard in computer vision for many years.
ROS2 already has OpenCV. You can follow tutorials online to see how to program with OpenCV in ROS2. e.g., https://www.theconstructsim.com/how-to-integrate-opencv-with-a-ros2-c-node/
● To keep things simple for your first steps, you can install OpenCV additionally in a virtual environment, so you can call it without going through ROS2.
Note: if you want the most options, download the contrib version.
For example, you could try follow the instructions here (e.g., (a) not (b)):
https://randomnerdtutorials.com/install-opencv-raspberry-pi/

For example, you could call your virtual environment `opencv-venv`

● Make a simple test script for yourself. For example you can make a file called `testWebCam.py` with code like this:

```
import cv2
cap = cv2.VideoCapture(0)
while(True):
          ret, frame = cap.read()
          if(ret):
                    img2 = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
                    edges = cv2.Canny(img2,100,200)
                    cv2.imshow('', edges)
                    key=cv2.waitKey(10) & 0xFF
                    if key == ord('q'):
                              print("Done")
                              break
                    elif key == ord('s'):
                              cv2.imwrite("screenshot.jpg",edges)
                              print("Took a Picture")
          else:
                    break

cap.release()
cv2.destroyAllWindows()
```

● Plug in your USB webcamera into an empty USB slot on the back of your rpi.

Open a terminal and paste in the following commands (or similar commands, depending on what you called your virtual environment and python file and where you put it, etc.):

```
cd Desktop
source opencv-venv/bin/activate
python testWebCam.py
```

Take a screenshot or photo of processing video feed and add it to your report.

4 ROS: make your own workspace and package. (~60 minutes)

Now we go back to ROS 2. In this course, you will need to know how to make your own ROS 2 packages for your robots.

Follow the tutorial to build a simple publisher and subscriber in your preferred language or both if you wish:

https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Publisher-And-Subscriber.html

https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Cpp-Publisher-And-Subscriber.html

Take a screenshot or photo or direct the console output to a file and add to your report.

5 Sending motion control commands to your robot (~5 minutes)

In this course, your robots will use a shared communication protocol to collaborate with other groups' robots. Let's check out an example of something you will be doing.

Modify your listener to listen on the channel "action".

Ensure you have two terminals set up, as before, for talking and listening.

Start the listener.

Now instead of running the talker from before, paste the following commands into the other terminal, representing commands to get a robot to move forward, backward, left, and right.

FWD:

```
ros2 topic pub -1 /action std_msgs/String "data: 1/9/2025-11:05:35.859906,g,-1,-1,-2,255;255"
```

BACK:

```
ros2 topic pub -1 /action std_msgs/String "data: 1/9/2025-11:05:35.859906,g,-1,-1,-2,-255;-255"
```

LEFT:

```
ros2 topic pub -1 /action std_msgs/String "data: 1/9/2025-11:05:35.859906,g,-1,-1,-2,0;255"
```

RIGHT:

```
ros2 topic pub -1 /action std_msgs/String "data: 1/9/2025-11:05.859906,g,-1,-1,-2,255;0"
```

FWD (HALF-SPEED):

```
ros2 topic pub -1 /action std_msgs/String "data: 1/9/2025-11:05:35.859906,g,-1,-1,-2,127;127"
```

STOP:

```
ros2 topic pub -1 /action std_msgs/String "data: 1/9/2025-11:05.859906,g,-1,-1,-2,0;0"
```

You should see the commmands in the listener terminal.
(What is this doing?
The first bit just publishes a string once to the channel action using ROS2.
The meaning of the second bit in quotes is described in the shared communication protocol doc linked on BB. Basically, the last two numbers are setting the speed for each motor, where 255 is full speed, 127 represents half-speed, 0 is stopped, and -255 is full speed in the opposite direction. We will use these commands in tollgate 2 to get the robots to dance, and you can use them also when you collaborate with other groups.)
Take a screenshot of sending ROS 2 motion commands and add it to your report.

Done!

Optional: Just for your own fun if you feel like it:

- try to send messages between two different machines (for example, between your group's RPI and another group's RPI); take a screenshot. You will need to do this anyway.
- try Vnc. This gives you a remote desktop, rather than just a command line through ssh. TigerVNC has worked nicely for me. You need the server running on your rpi, and a client viewer on your other computer.
- try moving files like images from your rpi to another computer with a USB drive, email, a program like Filezilla, or for linux, you can move to a nice directory on your computer then do something like: "sudo scp pi@192.168.1.X:/home/pi/test.jpg ."
- try out other tutorials for ROS2, etc. for example, try using OpenCV through ROS2. For example, you can adapt the tutorial at, changing the name of the distribution from foxy to jazzy: https://automaticaddison.com/getting-started-with-opencv-in-ros-2-foxy-fitzroy-python/ (a tip: try change the name in package.xml to python3-opencv; you can also check out test_ros2_ws if it helps)